a2kit User Manual

New here? Start with the Quick Start. This manual covers every command in detail.

Contents

1. Concepts
2. Setup
3. `a2kit image`: pictures
4. `a2kit audio`: sound
5. `a2kit video`: animation
6. `a2kit show`: picture-and-sound shows
7. `a2kit send`: upload one block
8. `a2kit stream`: play a video
9. Housekeeping: `ping`, `peek`, `crash`, `canary`, `ramload`, `disk`, `mon`, `build`, `test`, `doctor`
10. Troubleshooting
11. Reference: formats, memory map, protocol, limits

> **Status:** working on real hardware: a 1979 Apple II+ with a Super Serial Card II, a Floppy Emu, and Linux on a built-in serial port (tested setup and measured speeds). Everything is also covered by the simulator tests (`./a2kit test`). Timings quoted below come from those simulations.

---

1. Concepts

The idea. Your computer does the hard work (resizing, dithering, compressing). The Apple II only has to receive bytes and put them where they belong, which even a 1 MHz 6502 does comfortably.

What the Apple II can show. Hi-res graphics are 280×192 pixels, one bit each: black or white. Greys are made by patterns of dots (*dithering*). There are two screens (*page 1* and *page 2*). Sound comes from a one-bit speaker that can only click, so audio is played by timing the clicks (about 8000 samples a second, 4 bits each): crunchy but recognisable.

The pipeline.

```
your file --(convert)--> .a2b / .a2v file --(send)--> serial cable --> Apple II
```

Step	Commands	Does what
Convert	`image`, `audio`, `video`, `show plan`	Turns PNG/GIF/WAV into Apple II data. Needs no hardware.

Step	Commands	Does what
Send	`send`, `stream`, `show send`	Uploads converted data over the serial cable.
Receive	the *receiver* program on the Apple II	Writes incoming data into memory, then runs a program or shows the screen.

Files you will meet

Extension	Meaning	Made by
`.a2b`	One *block*: an image or audio data with an 8-byte header	`image`, `audio`
`.a2v`	A video *stream* (frames of changes, plus optional sound)	`video`
`.toml`	A show script	you
`.bin`	A built 6502 program (`receiver.bin`, `sequencer.bin`, `video.bin`)	`build`

Programs on the Apple II. The *receiver* (booted from the a2kit disk, or typed in once per power-up) listens to the serial card. The *sequencer* plays shows and the *video player* plays animations; `--install` uploads those for you through the receiver.

---

2. Setup

2.1 Your computer

Full instructions, with per-system commands and troubleshooting, are in the Installation Guide. In short:

1. Install Python 3.11 or newer.
2. `python3 -m pip install -r requirements.txt` (pillow, numpy, pyserial), preferably inside a virtual environment. Use `requirements-dev.txt` to add `py65` for the tests.
3. Optional: **cc65** and **make** to rebuild the 6502 programs (the built `.bin` files are included), and **ffmpeg** to convert video files such as `.mp4`. GIFs and folders of frames need no ffmpeg.
4. Run `./a2kit doctor`. It reports anything missing.

On Windows, run commands as `python tools\a2kit.py COMMAND …` in place of `./a2kit COMMAND …`.

2.2 The Apple II

You need a 48K Apple II, II+ or IIe running at its normal 1.02 MHz, with a **Super Serial Card in slot 2**, connected to your computer's RS-232 serial port (built in, or a USB-to-RS-232 adapter). The full list, including the IIc, the IIgs, speed, memory and the cable, is under Hardware in the Installation Guide. Everything here assumes slot 2. The slot is set in `runtime/config.inc`: change `SLOT`, run `./a2kit build` (needs cc65), and boot the new disk or retype the new `receiver.bin`.

The serial settings are 115200 baud (or 19200, see 2.4), 8 data bits, no parity, 1 stop bit. The receiver sets these in software, so the card's baud DIP switches don't matter to a2kit; the card's wiring jumpers (which swap the transmit and receive lines) do. See Troubleshooting if nothing arrives.

2.3 Start the receiver on the Apple II (once per power-up)

Option A: boot the a2kit disk. `a2kit-boot.dsk` is a 140 KB 5.25-inch disk image (DOS 3.3 sector order) that boots straight into the receiver. There is no DOS on it, so the memory DOS would use (`$9600–$BFFF`) stays free for sound. See the Installation Guide for getting it onto a floppy, a Floppy Emu or an emulator.

1. Connect the cable, put the disk in drive 1 of the disk controller (normally slot 6), and power on, or type `PR#6` at the BASIC prompt.
2. The screen clears and shows `A2KIT RX READY 115200 (ANY KEY QUITS)`. The number is the line speed; `a2kit-boot-19200.dsk` shows `19200` (see 2.4).

The booted receiver **restarts itself after every session**: a picture, a show or a video ends and it is listening again on a freshly cleared screen, with no typing. A key pressed while it is waiting drops you into the Monitor; `980G` (or `CALL 2432` from BASIC) starts it again. It needs a 16-sector Disk II controller (the kind sold from DOS 3.3 onwards); an original 13-sector controller can't boot it.

Option B: type it in.

1. `./a2kit mon receiver.bin` prints 47 lines of Monitor input (`0800:78 A9 00 …`) and a final `0800G`. For 19200 baud, use `receiver-19200.bin`.
2. On the Apple II, reach the Monitor prompt `*` (from BASIC: `CALL -151`).
3. Type the 47 address lines, pressing Return after each. Leave out the `0800G` for now.
4. Verify: type the range `mon` printed at the end (`0800.0975` for `receiver.bin`) and compare the dump against what you typed.
5. Connect the cable and type `0800G`. You should see `A2KIT RX READY 115200 (ANY KEY QUITS)`.

Typed in, the receiver **exits after each upload session** (after a picture, a show or a video), so type `0800G` again before the next one. To get the boot disk's restart-by-itself behaviour on a typed-in receiver, also type `0980:20 2F FB 20 58 FC 20 00 08 90 F5 4C 69 FF` and start it with `980G` instead of `0800G`.

Things to know (both options):

- Pressing a key on the Apple II while the receiver is idle quits it.
- The receiver lives at `$0800–$0AFF`, which is where Applesoft BASIC keeps programs. Don't load or run a BASIC program between sessions, or you'll have to retype it.
- It is happiest on a machine booted straight to the Monitor with no disk system loaded. DOS and ProDOS use memory at the top of RAM that audio also needs.

2.4 Line speed: 115200 or 19200 baud

a2kit comes with two receivers and two boot disks. Everything else (the sequencer, the video player and your converted files) works with either.

Disk	Receiver	Banner	Speed
`a2kit-boot.dsk`	`receiver.bin`	`A2KIT RX READY 115200`	115200 baud, the default. Measured on the tested setup, a picture uploads in 0.9 s instead of 4.8, the starter show in 2.9 s instead of 15, and video gets six times as many bytes per frame
`a2kit-boot-19200.dsk`	`receiver-19200.bin`	`A2KIT RX READY 19200`	19200 baud: the fallback, and the fastest the 6551's own baud-rate generator gives

At 115200 the receiver turns the card's baud-rate generator off and runs straight from its 1.8432 MHz crystal (crystal ÷ 16 = 115200), the same setting ADTPro uses on an unmodified Super Serial Card. At that speed a byte arrives every 88.6 cycles of the Apple II's clock and the card holds only one, so the receiver and the video player must read each byte before the next one lands. The simulator tests check that cycle by cycle: the receiver needs about 63 cycles per byte and the video player about 67.

Tell the computer which speed the receiver uses, since the two ends must match. 115200 is the default. For the 19200 disk, either pass `--baud 19200` to `ping`, `send`, `show`, `stream` and `video`, or set it once:

```
export A2KIT_BAUD=19200          # Linux/macOS; on Windows: set A2KIT_BAUD=19200
./a2kit ping --port /dev/ttyS0   # now at 19200
```

Start with `./a2kit ping`: at 115200 it should say `Link OK`, with a speed-test rate of about 8,700 bytes/s on a port like the tested one (about 76% of the line rate: even at the 0.6 ms per packet measured at this speed, the port is a bigger share of each packet's time; `--chunk 255` uploads claw some of it back, reaching about 9,150 bytes/s, 79%). If it reports damaged packets, try a shorter cable or use the 19200 disk. A cable and card that already work with ADTPro at 115200 are a good sign.

Video: convert with the speed you'll play at (add `--baud 19200` for the 19200 disk), since the byte budget per frame comes from it. A clip converted for 19200 plays fine at 115200; one converted for 115200 runs late at 19200 (`stream` warns you).

3. `a2kit image`

What it does: converts any picture to a 280×192 black-and-white hi-res screen, ready to display.

When to use it: title cards, still pictures, and checking how artwork will dither. `--preview` shows you the result without any hardware.

Steps

1. Pick a picture. High contrast and bold shapes work best; fine detail turns to noise at 280×192.
2. Convert it and write a preview:

```
./a2kit image examples/logo.png -o out/logo.a2b --preview out/logo_preview.png
```

3. Open the preview. If it's too dark, add `--invert`. If it has black bars and you want it to fill the screen, add `--crop`.
4. To see it on the Apple II: `./a2kit send out/logo.a2b --port /dev/ttyUSB0` (with the receiver running).

Usage: `./a2kit image INPUT -o OUTPUT [options]`

Option	Meaning
`-o`, `--output FILE`	Where to write the block (required). Use `.a2b`.
`--dither atkinson` / `threshold`	`atkinson` (default) spreads shading into dots and looks photographic. `threshold` makes pure black and white, best for line art and text.
`--crop`	Scale and trim to fill the whole screen. Default is to fit the whole picture and fill the rest with black.
`--invert`	Swap black and white.
`--preview FILE`	Also write the dithered 280×192 result as a PNG.
`--raw`	Write the bare 8192-byte screen with no header (for your own programs).

Output: an 8200-byte `.a2b` (8-byte header + 8192-byte screen). The screen is stored in the Apple II's interleaved memory order, so it can be copied straight to `$2000`.

Tips: the picture is converted in grey, so colour is lost. White-on-black looks best on a green or white monitor; on a colour TV, fine dot patterns can show coloured fringes (an artefact of how the Apple II makes colour).

---

4. `a2kit audio`

What it does: converts a WAV file to the format the Apple II's speaker player uses: 4-bit samples at about 7972 Hz, two per byte, padded to 256-byte pages.

When to use it: to check how long and how big your sound will be, and to produce raw sample data for your own programs. You **don't** need to run it before using sound in a show or video, because those commands read your `.wav` file directly.

Steps

1. Choose a short WAV (8- or 16-bit, any sample rate, mono or stereo; stereo is mixed down).

2. Convert it:

```
./a2kit audio examples/jingle.wav -o out/jingle.a2b
```

3. Read the line it prints, for example `wrote out/jingle.a2b: 6920 bytes, 1.7 s`. If it's over about 6 seconds it won't fit in a show or video.
4. If it's too quiet or too loud, adjust `--gain`. If it's too long, use `--max-seconds`.

Usage: `./a2kit audio INPUT -o OUTPUT [options]`

Option	Meaning
`-o`, `--output FILE`	Output block (required).
`--gain N`	Multiply the volume after the sound has been normalised (default 1.0). Values above 1 clip.
`--max-seconds N`	Keep only the first N seconds.
`--no-dither`	Turn off the noise added before quantising to 4 bits. It's usually left on because it hides the stepping.
`--raw`	Write the bare samples with no header.

How it sounds: the sound is low-pass filtered, resampled to about 8 kHz, normalised to near full scale and squeezed to 16 levels. Speech and simple tones work; dense music turns to hiss.

Limits: one block holds at most 65,280 bytes (about 16 s), but a show or video has room for only about 6 seconds in total (see memory map).

---

5. `a2kit video`

What it does: turns an animation into a *delta stream*: for each frame, only the screen bytes that changed. It can also add sound.

Why deltas: a full screen is 8 KB but the cable carries about 1.9 KB a second at 19200 baud (about 11.5 KB at 115200). So each frame gets a small byte budget (about 300 bytes at 6 frames a second, about 1,800 at 115200), spent on the changes that matter most first. Anything that doesn't fit stays pending and is sent on later frames, so the picture catches up rather than losing detail for good.

Steps

1. Prepare the animation as a **folder of frame images** (played in name order, with numbers sorted as numbers: `f2.png` comes before `f10.png`), an **animated GIF**, or a **video file** (needs ffmpeg). A plain dark background with a small moving subject works best; full-screen motion smears.
2. Convert:

```
./a2kit video examples/frames -o out/bounce.a2v --fps 6
```

3. Read the summary line: frames, bytes, how long it will take to play, and **unsent screen bytes after the last frame**. That last number should be 0 (the picture fully caught up). If it isn't, the clip is too busy: lower `--fps` (more bytes per frame), slow the motion, or shrink the moving part.
4. Play it: start the receiver, then `./a2kit stream out/bounce.a2v --port /dev/ttyUSB0 --install`.

With sound

```
./a2kit video examples/frames -o out/bounce_sound.a2v --fps 6 --audio examples/boing.wav
```

The speaker player needs the whole CPU and can't listen to the cable while it runs, so sound can't play *during* video. Instead the sound is split into short bursts that play between frames while the computer stays silent for a moment. Nothing is dropped, but:

- the sound plays **in order, chopped into chunks** spread through the clip; it is not lined up with specific frames;
- the show runs **longer than the video alone**, by the length of the sound plus a short pause (0.05 s) per burst; the summary line gives the total;
- the sound is uploaded first and lives in memory, so clips with sound **need `--install`** when streamed, and the whole sound must fit in about 6 seconds.

Usage: `./a2kit video INPUT -o OUTPUT [options]` where INPUT is a folder, a GIF or a video file.

Option	Meaning
`-o`, `--output FILE`	Output stream (required). Use `.a2v`.
`--fps N`	Playback frames per second, a whole number from 1 to 60 (default 6). Higher fps means fewer bytes per frame.
`--src-fps N`	The frame rate of your *input*, if different (default: same as `--fps`). Example: a folder shot at 24 fps played at 6 fps is `--src-fps 24 --fps 6`, which keeps every 4th frame.
`--baud N`	The link speed used to work out the byte budget: 115200 (default) or 19200 (2.4).
`--budget N`	Set the bytes-per-frame budget directly, overriding the baud-based default.
`--dither ordered` / `atkinson` / `threshold`	`ordered` (default) gives stable dots that don't shimmer between frames and keep the changes small. `atkinson` looks better on stills but flickers in motion.
`--crop`	Fill the screen instead of letterboxing.
`--max-seconds N`	Use only the first N seconds.
`--scrub N`	Re-send N screen bytes every frame in a rotating sweep, which repairs damage after a lost byte (a full sweep takes 8192÷N frames). Off by default; 8–16 is cheap insurance on a shaky link.

Option	Meaning
`--audio FILE.wav`	Add sound, played in bursts between frames.
`--burst-pages N`	Length of each sound burst in 64 ms units (default 1). The encoder raises it if there would be more bursts than frames.

Notes

- GIF frame timings are ignored: every frame is played at `--fps`.
- The screen starts black, so a dark background costs nothing, while a bright or noisy one takes many frames to paint at the start.
- Video files need `ffmpeg` on your PATH. Tested with an H.264 `.mp4` made from the example frames; frame folders and GIFs are tested too. Other video formats go through the same ffmpeg step but haven't been tried.

---

6. `a2kit show`

What it does: builds a *show*, an ordered list of scenes (pictures, sounds, pauses) described in a small text file, and sends it to the Apple II, where the *sequencer* plays it.

Two commands

- `./a2kit show plan SHOW.toml` converts everything and prints what would be uploaded. Needs no hardware.
- `./a2kit show send SHOW.toml --port PORT [--install]` uploads and starts it.

Steps

1. Copy the starter script: `cp examples/show.toml myshow.toml`.
2. Edit it. Paths are relative to the script file.
3. Check it: `./a2kit show plan myshow.toml`. It prints one line per piece (address, size, what it is), the total size and the estimated upload time.
4. Make sure the receiver is listening (boot disk: it is; typed in: `0800G`).
5. Send it: `./a2kit show send myshow.toml --port /dev/ttyUSB0 --install`. Use `--install` the first time after a power-up; later shows don't need it as long as the sequencer is still in memory. It names each piece as it uploads it, then reports how long the upload took (about 3 s for the starter show at 115200 baud, about 15 s at 19200).
6. Watch. Press any key to stop between scenes (sound cannot be interrupted once it starts).

The script format

```toml
[[scene]]
image = "logo.png"     # show this picture (page 1 first, then page 2)
hold  = 3.0            # optional: stay on it for 3 seconds

[[scene]]
audio = "jingle.wav"   # play this sound over whatever is on screen
gain  = 1.0            # optional volume

[[scene]]
wait = 2.0             # pause for 2 seconds
```

Scene key	Meaning
`image`	A picture, converted like `a2kit image` (letterboxed, Atkinson dither). The first image goes to page 1, the second to page 2.
`hold`	Only with `image`. Seconds to wait after showing it.
`crop`, `invert`, `dither`	Only with `image`. Same meaning as the `a2kit image` options: `crop = true`, `invert = true`, `dither = "threshold"` (default `"atkinson"`).
`audio`	A WAV file, played when this scene is reached.
`gain`	Only with `audio`. Volume multiplier (default 1.0).
`wait`	Seconds to do nothing. Times are in steps of 0.1 s.

Each `[[scene]]` has exactly one of `image`, `audio` or `wait`, and only the keys listed for it. A misspelled or misplaced key is an error that names the key, so typos don't pass silently.

When the show ends the screen goes back to text. If you want the last picture to stay up for a while, give it a `hold` or add a `wait` after it.

Limits

- At most **two** image scenes (one per hi-res page). Showing a third picture isn't supported.
- Sound totals about **6 seconds** across the show.
- About 40 scene entries (a `hold` counts as one extra).
- If you break a limit, `plan` stops with a message such as `only two image scenes fit` or `audio doesn't fit below $C000`.

Usage: `./a2kit show plan|send SHOW.toml [options]`. Options: `--port`, `--baud` (default 115200, or `A2KIT_BAUD`), `--install`, `--rtscts`.

> If you send a show **without** `--install` and the sequencer isn't in memory, the Apple II will jump into empty memory and misbehave. After a power cycle always use `--install`.

7. `a2kit send`

What it does: uploads one file to the Apple II and, by default, displays it. The workhorse for single pictures, and for loading any data to any address.

Steps (a picture)

1. Convert: `./a2kit image examples/logo.png -o out/logo.a2b`.
2. Make sure the receiver is listening (boot disk: it is; typed in: `0800G`).
3. Send: `./a2kit send out/logo.a2b --port /dev/ttyUSB0`. A progress counter runs while it uploads, and the picture appears at the end.
4. Press a key on the Apple II to return to text.

Sending other things

```
./a2kit send mydata.bin --port /dev/ttyUSB0 --raw --addr 0x6000       # raw bytes to $6000
./a2kit send prog.bin   --port /dev/ttyUSB0 --raw --addr 0x6000 --exec 0x6000   # ...then run it
```

Usage: `./a2kit send FILE --port PORT [options]`

Option	Meaning
`--port PORT`	Serial port (required), e.g. `/dev/ttyUSB0`, `COM3`.
`--baud N`	Line speed: must be the number in the receiver's banner, 115200 (default) or 19200. `A2KIT_BAUD` sets the default.
`--addr A`	Load address, like `0x2000`. Optional for picture blocks (they load to `$2000`); required for anything else.
`--exec A`	When the upload finishes, run the code at this address. The default, 0, instead shows hi-res page 1 until a key is pressed.
`--raw`	Send the file as-is, with no a2kit header (needs `--addr`).
`--keep-header`	Also send the 8-byte a2kit header.
`--chunk N`	Bytes per packet, 1–255 (default 128). Smaller packets recover faster on a noisy link.
`--retries N`	Resends per packet before giving up (default 5).
`--rtscts`	Turn on hardware flow control on your computer's port. Leave off unless your cable needs it.

How it protects your data: each packet carries an address and a checksum, and the Apple II replies ACK or NAK. A bad packet is resent and never written to memory.

Protected memory: `send` refuses (before opening the port) any destination that would overwrite zero page and the stack (`$0000–$01FF`), the receiver and its buffer (`$0800–$0AFF`), or I/O and ROM (`$C000–$FFFF`).

Not for videos: `.a2v` files are played with `stream`.

---

8. `a2kit stream`

What it does: plays a `.a2v` video on the Apple II.

Steps

1. Convert a clip with `a2kit video` and check that it reports 0 unsent bytes.
2. Make sure the receiver is listening (boot disk: it is; typed in: `0800G`).
3. Run: `./a2kit stream out/bounce.a2v --port /dev/ttyUSB0 --install`.
4. `--install` uploads the video player (and the sound, if the clip has any), starts it, and then streams the frames. The Apple II clears the screen and the animation paints in; the first moments may look rough while the screen fills.
5. The video ends by itself. Press any key on the Apple II to stop early.

Usage: `./a2kit stream FILE --port PORT [options]`

Option	Meaning
`--port PORT`	Serial port (required).
`--baud N`	Line speed: the number in the receiver's banner, 115200 (default) or 19200. `A2KIT_BAUD` sets the default.
`--install`	Upload the video player (and any sound) first. Use it every time unless the player is already running. **Required** for clips with sound.
`--rtscts`	Hardware flow control on your port.

Things to know

- After the player starts there is **no flow control and no per-packet acknowledgement**. The computer therefore keeps its own model of the serial line: it sends each frame at its due time but never before the previous one has finished crossing the cable, and after a frame that ends in a sound burst it stays silent until the burst is over. It doesn't rely on the operating system or the USB adapter saying when bytes have left, which they don't report reliably.
- The player checks the stream as it goes, discards a damaged frame and carries on at the next one, but it can't ask for a resend. Use `--scrub` when you encode if your link is shaky.
- If `stream` warns that frames run behind, the clip's byte budget is too high for your link: re-encode with a lower `--fps` or `--budget`.
- For clips with sound, the computer pauses for each sound burst, so total running time is the video length plus the sound length.

---

9. Housekeeping: `ping`, `peek`, `crash`, `canary`, `ramload`, `disk`, `mon`, `build`, `test`, `doctor`

`a2kit ping`

Checks the serial link to a running receiver and measures it. Run it before your first upload, and whenever uploads fail.

Steps

1. Start the receiver on the Apple II (boot the disk, or `0800G`). The banner must be on screen.
2. Run:

   ```
   ./a2kit ping --port /dev/ttyUSB0
   ```

3. Watch the Apple II's bottom line: each ping writes `A2KIT PING 1 OF 10`, `2 OF 10` and so on there. Seeing it count up proves the data really arrives in the Apple II's memory, not just that something answers.
4. Read the summary and the plain-language verdict at the end (below).

ping never ends the session, so the receiver keeps listening and you can `send`, `show send` or `stream` straight afterwards. It writes to the bottom text line (`$07D0–$07F7`) and to the first `--chunk` bytes of hi-res page 2 (`$4000`), so don't run it between uploading data there with `send` and using it.

Before the pings, ping switches the port to **low-latency mode** where the driver offers it, and says `(low-latency mode on)` on its first line when it did. On Linux this turns an FTDI USB adapter's 16 ms latency timer down to 1 ms with no `sudo`, and the setting lasts until the adapter is unplugged, so running ping once after plugging it in speeds up later uploads too. On a built-in port it changes nothing measurable; on Windows and macOS it does nothing.

What it reports

Each ping prints its round trip, split into two parts:

- **line and Apple II:** the time the bytes need on the cable at the `--baud` speed, plus the receiver's work. This part is fixed.
- **port:** whatever is left. This is the delay your computer's serial port adds before the reply reaches the program.

After the pings, a **speed test** sends `--speed` full packets and reports the real upload rate, as bytes per second, as a share of the line rate (1,920 bytes/s at 19200 baud, 11,520 at 115200), and as the time an 8 KB picture would take.

The verdict at the end is one of these:

Verdict	Meaning
`Link OK`	Every packet was acknowledged. If your port adds more than 5 ms per packet, it also says how much faster uploads would be without that, judged from the speed test (see below).
`No reply at all`	Nothing came back. It lists what to check, in order: the receiver, the port name, the cable and jumpers, the adapter type, the slot.
`Bytes come back, but not the receiver's replies`	Usually a speed mismatch: `--baud` must be the number in the receiver's banner (115200, or 19200 with `a2kit-boot-19200.dsk`). Otherwise wiring or signal levels.
`some packets arrive damaged`	NAKs: the Apple II got packets that failed their checksum. Noise, a long or poor cable, or a marginal adapter.
`Some packets got no reply`	A loose connection, or the receiver stopped (a key pressed on the Apple II).

The exit code is 0 when every packet was acknowledged, 1 otherwise, so ping can be used in scripts.

Options: `--port` (required), `--baud` (default 115200, or `A2KIT_BAUD`), `--count` pings (default 10), `--speed` speed-test packets (default 16; 0 skips it), `--chunk` bytes per speed-test packet (1–255, default 128), `--timeout` seconds to wait for each reply (default 1.0).

If the port delay is high

Uploads wait for a one-byte reply after every packet, so a port that holds replies back slows every upload (video streaming has no replies and isn't affected). Typical causes and fixes:

Port	Typical delay	What to do
USB adapter with an FTDI chip	Up to 16 ms (its "latency timer"); 11 ms measured on a tested FT232R at 115200, and 1.1 ms with the timer at 1 ms	Linux: `a2kit ping` sets the 1 ms timer itself, until the adapter is unplugged. By hand: `echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer`. Windows: Device Manager → the port → Port Settings → Advanced → Latency Timer: 1 ms.
Built-in serial port (16550-type)	0.6 ms measured at 115200 and 2.2 ms at 19200 on Linux (a lone reply waits for the chip's FIFO timeout)	Linux, where the kernel offers it: `echo 1 | sudo tee /sys/class/tty/ttyS0/rx_trig_bytes`
Other USB adapters (CH340, CP210x, PL2303)	Usually 1–3 ms	Little to tune.

Larger packets also help, since each one pays the delay once: `./a2kit send --chunk 255`. The exception is an FTDI adapter still on its 16 ms timer: every packet then takes a whole number of 16 ms periods, and at

115200 a 255-byte packet takes two where a 128-byte one takes one, so the rate stays about the same (see the measurements in the Installation Guide).

`a2kit peek`

Reads Apple II memory through the running receiver and prints it like the Monitor's dump.

```
./a2kit peek BD00 16 --port /dev/ttyUSB0
./a2kit peek 2000 8192 --port /dev/ttyUSB0 -o screen.bin
```

Addresses are in hex, and the length is 16 bytes if you leave it out. `-o` also saves the bytes to a file. `$C000–$CFFF` is refused, because just reading a soft switch can change the machine's state. `$D000–$FFFF` reads ROM, not a language card.

`a2kit crash`

If a program hits `BRK`, the receiver keeps the Monitor's saved registers, prints them on the screen as usual, and goes on listening instead of stopping at the `*` prompt. A key still drops to the Monitor.

```
./a2kit crash --port /dev/ttyUSB0            # report any crash since the receiver started
./a2kit crash --port /dev/ttyUSB0 --watch    # stay connected and log a crash when it
happens
```

The report gives the registers, the vectors at `$03F0`, the whole stack page, and whether the receiver's own code still matches `receiver.bin`. `--watch` checks every 30 seconds (`--interval` changes that), logs each crash with its time, and also logs when the Apple II stops answering, which is all you'll see from a hang or a crash that never reaches `BRK`. It keeps the port open until you stop it.

Both commands need the receiver running from a disk built with these features. An older one just doesn't answer. A marker sent while nothing has the port open is lost, but the report is still there to read later.

The receiver sends read replies back-to-back, so these commands may not work on a card refitted with a WDC 65C51, which always reports its transmitter as ready.

`a2kit canary`

Fills free RAM with a known pattern, then reads it back every few minutes and logs any byte that changes. Use it on a machine that crashes or hangs when left alone, to find RAM that loses its contents or writes that land in the wrong place.

```
./a2kit canary --port /dev/ttyUSB0              # fill, then check every 5 minutes
./a2kit canary --port /dev/ttyUSB0 --no-fill    # go on checking a region an earlier run
filled
```

It fills `$0B00–$BFFF`, the 45 KB that the receiver, the Monitor and the text screen don't use, which takes about 5 seconds at 115200. Each byte's value comes from its address, so neighbouring bytes differ, and so do bytes at the same place in another 16K bank. Each check logs:

- **`CANARY:`** with each newly changed byte (up to 40 listed), what it should be, what it is now, and the xor of the two, then a `bits changed:` line summing up which data bits were involved.
- **`back to the pattern:`** for bytes that changed earlier but now read correctly again. Bytes that come and go like this point at a chip that can't hold its data, rather than a single stray write.
- **`STACK PAGE:`** when the number of zero bytes in `$0100–$01FF` jumps by 16 or more, as in a stack wipe.

Between checks it watches the receiver like `crash --watch`, logging crashes and silences, and it checks memory straight away when the Apple II answers again after a silence. It also logs one line an hour, so a quiet log shows it's still running. `--check` sets the seconds between memory checks (default 300) and `--interval` the seconds between crash checks (default 30).

Reading the result. On a II+ (and the original Apple II), the 48K is three rows of eight 4116 chips: one row for each 16K bank (`$0000`, `$4000` and `$8000`), and one chip in each row for each data bit. A IIe has a single row of eight 64K chips, so there the changed bit alone names the chip. So if every change is in one bank and one bit, for example only `D0` in `$8000–$BFFF`, suspect that chip or its socket. Swap it with its neighbour in the same row and run again. If the errors move to the next bit, it's the chip; if they stay, it's the socket. Don't swap chips into the `$0000–$3FFF` row, which holds the receiver. Changes scattered across banks and bits point elsewhere, such as at an address fault or the power supply.

- It keeps the port open until you stop it with Ctrl-C, so nothing else can use the receiver meanwhile.
- The fill overwrites both hi-res pages and anything else you loaded into that region.
- A fault that takes hours to appear needs a run of hours or days. On the tested machine, a failing chip first showed after 8 h 42 min.

`a2kit ramload`

Starts a new build of the receiver on a running one, over the serial line, so you can try it without copying a disk image to an SD card or rewriting a floppy.

```
./a2kit build
./a2kit ramload --port /dev/ttyUSB0
```

The new receiver restarts as if it had just booted: the screen clears and shows its banner. If it can answer read requests, the tool then checks the code in memory against the image and says whether it matches.

This only changes memory. **A reboot or power-off brings back whatever is on the disk**, so copy the new `a2kit-boot.dsk` to the disk when you want to keep it.

- The receiver must be listening, so close anything else using the port first, such as `a2kit crash --watch`.

- It overwrites `$2000–$21FF` and `$6000–$601A`, and it can't start over a crashed machine that has stopped answering.
- `--boot FILE` loads another 512-byte boot image. Use one built for the speed you're running at; `--baud 19200` picks `boot-19200.bin` by default.
- `--disk-slot` names the disk card's slot, if it isn't 6. The boot code uses it to turn off that drive's motor.

`a2kit disk`

Makes the boot disk image. Ready-made `a2kit-boot.dsk` (115200 baud) and `a2kit-boot-19200.dsk` come with a2kit, so you only need this after rebuilding the receivers (for example after changing the slot in `runtime/config.inc`; `./a2kit build` remakes both images too).

```
./a2kit disk                      # writes a2kit-boot.dsk
./a2kit disk --baud 19200         # writes a2kit-boot-19200.dsk
./a2kit disk -o myboot.dsk        # another name; .po works too
```

How it boots: the Disk II boot ROM loads two sectors of track 0, the boot code turns the drive motor off, points output and input back at the screen and keyboard (booting with `PR#6` leaves them on the disk card), moves the receiver to `$0800` and starts it in a loop at `$0980`, which resets and clears the text screen before every start, so you always see one fresh banner. The second boot sector is written to all of sectors 1–15 of track 0, so the image boots whether a tool treats it as DOS order (`.dsk`, `.do`) or ProDOS order (`.po`). Only track 0 is used; there is no DOS and no catalog, so DOS can't read the disk as a file disk.

`a2kit mon`

Prints a binary as lines you can type at the Apple II Monitor.

```
./a2kit mon receiver.bin              # starts at $0800
./a2kit mon prog.bin --addr 0x6000    # starts at another address
./a2kit mon receiver.bin > receiver.txt
```

Each line is `ADDR:` and eight hex bytes. The last line is `ADDRG`, which runs the program. Use this once per power-up for `receiver.bin` if you aren't using the boot disk. The bigger programs (`sequencer.bin`, `video.bin`) are uploaded for you by `--install` and aren't meant to be typed in.

`a2kit build`

Assembles the 6502 programs (`receiver.bin` and `receiver-19200.bin`, `sequencer.bin`, `video.bin`) and both boot disks from the sources in `runtime/`. Needs cc65 and make. Built copies are already included, so you only need this if you change the sources or the slot in `runtime/config.inc`.

`a2kit test`

Runs the whole test suite: converters, protocol, the receiver, the sequencer, video playback with fault injection, the command line and the examples and Quick Start. Needs cc65 and `py65`. Takes under a minute.

`a2kit doctor`

Checks that cc65, the Python packages and the built `.bin` files are present, and tells you what to install. Run it first whenever something looks off.

---

10. Troubleshooting

Symptom	Likely causes and fixes
`no ACK after 5 tries` right at the start	Run `./a2kit ping` first: it tells "no reply", damaged packets and a speed mismatch apart. Causes: the receiver isn't running (the banner must be on screen: boot the disk, or type `0800G`; `980G` restarts a booted one). Wrong port name. The cable needs a null-modem adapter, or the card's jumper block is in the other position (the lines are swapped). The card isn't in the slot set in `runtime/config.inc` (2 unless you changed it). A typo in the typed-in receiver (compare the range `mon` printed, `0800.0975`, with the printed lines). Speeds that don't match: `--baud` must be the number in the banner.
Receiver banner never appears	Typed in: `0800G` didn't run; recheck the typed lines. A BASIC program or disk system overwrote `$0800`; reboot and start again.
Boot disk: the drive spins or the screen fills with junk	The image wasn't written as a 140 KB 5.25-inch disk (check the tool's format setting), the controller is a 13-sector one, or the disk isn't in drive 1. Try the image in an emulator first.
Uploads work but with many retries	`./a2kit ping` shows NAKs. Noisy or long cable. Try `--chunk 64`. Use the 19200 disk first. To go lower still: in `runtime/config.inc` set `BAUD = 9600`, run `./a2kit build`, boot the new `a2kit-boot.dsk`, and pass `--baud 9600` (or set `A2KIT_BAUD=9600`).
`show send` hangs or the machine misbehaves	Forgot `--install` after a power-up. Or something else is using memory at `$0B00–$1000` or `$6000–$BFFF`.
`sequencer.bin not found`	Run `./a2kit build` (needs cc65).
Sound is too high-pitched and pauses are too short	The machine runs faster than 1.02 MHz (an accelerator card, a IIgs in Fast mode, a IIc Plus at full speed). Set it to normal speed.

Symptom	Likely causes and fixes
Nothing ever arrives, even with the receiver running	The adapter is a USB-TTL board rather than RS-232, or (on a IIgs) you're using the built-in serial port, which a2kit can't drive. See Hardware.
Picture shows with coloured fringes or stripes	An artefact of Apple II hi-res on colour displays. Use a monochrome or green monitor, or the monitor's colour-off setting.
Picture too dark or too bright	Try `--invert`, `--dither threshold`, or adjust the source image's contrast. Check `--preview`.
Video starts messy	Normal: the first frames paint a full screen over several seconds. Use a darker, simpler start.
Video smears or lags	The clip is busier than the cable allows. Lower `--fps`, slow the movement, shrink the moving part, or use a plain background. Make sure `unsent screen bytes after last frame` is 0.
`warning: frames run up to X s behind`	Lower `--fps` or `--budget` and re-encode.
`refusing to send: … would overwrite …`	The destination overlaps protected memory (see `send`). Pick another `--addr`.
`a2kit COMMAND: error: …`	A one-line description of what went wrong (a missing file, a bad `show.toml`, the port not opening). Set `A2KIT_DEBUG=1` to see the full Python traceback.
Small specks left on screen after a glitch	Re-encode with `--scrub 8` (or 16).
Sound is chopped	That's how sound works alongside video. Use a show (picture + sound) if you want a continuous sound.
`only two image scenes fit`	A show has two hi-res pages. Combine pictures or split into two shows.
`audio doesn't fit below $C000` / `only 96 (6 s) fit`	Shorten or trim the sound (`--max-seconds`).
`this clip has audio … use --install`	Add `--install` to the `stream` command.
`ffmpeg isn't on PATH`	Install ffmpeg, or convert to a GIF or folder of frames.
`ModuleNotFoundError`	Activate your virtual environment and run `python3 -m pip install -r requirements.txt`. Python 3.11+ is required. See the Installation Guide.

If the cause isn't clear, narrow it down in order: `./a2kit doctor` (your computer's setup), then `./a2kit ping` (the cable and the receiver; its verdict says which kind of fault), then `./a2kit test` (confirms your copy of a2kit works), then send a single small picture with `--chunk 32`, then try a lower baud rate.

---

11. Reference

11.1 File formats

Block header (`.a2b`, 8 bytes, little-endian)

Offset	Size	Field
0	2	Magic `A2`
2	1	Format version (currently 1; the programs refuse a mismatch)
3	1	Block type: 1 hi-res image, 2 double hi-res image (reserved), 3 video, 4 audio, 5 effect (reserved)
4	2	Payload length
6	2	Cycle budget per frame (audio blocks: 128, the speaker player's cycles per sample)

Video stream (`.a2v`): an 8-byte header whose length field holds the *frame count*, one byte of fps, then for each frame a 2-byte length and the frame bytes. Optionally `AU`, a 2-byte length and the sound, which `stream` uploads before starting.

Frame commands (what the Apple II player understands). Every data byte is below `$80` and every control byte is `$80` or above, which is how the player notices damage:

Byte	Meaning
`$FE`	Start of frame (also the resynchronisation mark)
`$01–$7E n`	Literal run: n data bytes follow, written at the current screen position
`$81–$FD`	Skip 1–125 screen bytes
`$FF lo hi`	Long skip of lo + 128×hi bytes
`$7F n c`	Play n sound pages (c = n xor `$55` as a check)
`$80 $80`	End of video

11.2 Memory map

Address	Used for
`$0800–$0975`	Receiver (374 bytes; 373 for the 19200 one)
`$0980–$09B2`	Boot disk only: the restart loop (`$0980`) and the boot code

Address	Used for
`$0A00–$0AFF`	Receiver's packet staging buffer
`$0B00–$0BFF`	Show scene table
`$0C00–$0DFF`	Sequencer (code, then the speaker routine)
`$0E00–$0FFF`	Video player
`$1000–$10FF`	Speaker routine used by the video player
`$2000–$3FFF`	Hi-res page 1
`$4000–$5FFF`	Hi-res page 2
`$6000–$BFFF`	Sound data (96 pages of 256 bytes, about 6.2 s)

Zero page `$06–$0F`, `$CE` and `$EB–$EF` are used by these programs, and the receiver points the Monitor's BRK vector (`$03F0–$03F1`) at its crash handler. The boot disk changes nothing else: the Monitor's I/O hooks are reset to the screen and keyboard, and the rest of memory is untouched. The sequencer and the video player switch interrupts off while they run (their sound is cycle-timed) and restore the previous setting when they return.

11.3 Upload protocol (`send`, `show send`, `--install`)

Packet from computer: `$A5 seq len addr_lo addr_hi data… checksum`. The checksum is the XOR of every byte after the `$A5`. The Apple II answers `$06` (ACK) or `$15` (NAK). A packet with `len` 0 ends the session; its address is what to run (0 means show hi-res page 1). The receiver holds each packet in a buffer and only copies it into memory after the checksum passes, so a corrupted address can't overwrite anything.

A read request is `$A6 count 0 addr_lo addr_hi checksum`, with the checksum over the four bytes after the `$A6` and a `count` of 0 meaning 256. The Apple II answers `$06`, the bytes, then their XOR, or `$15` if the checksum is bad.

If a program hits `BRK`, the Monitor saves the registers (PC at `$3A–$3B`; A, X, Y, P and S at `$45–$49`) and the receiver's handler sends `$C3`, sets `$CE` to `$C3`, prints the registers and goes on listening. A key then drops to the Monitor. Starting the receiver again clears `$CE`.

11.4 Limits at a glance

Thing	Limit
Apple II	48K II, II+ or IIe at 1.02 MHz with a Super Serial Card in slot 2 (details)
Screen	280×192, black and white, two pages
Sound in a show or video	about 6 seconds in total
Sound quality	4 bits at about 7972 Hz (one-bit speaker)

Thing	Limit
Images in a show	2
Video frame size	about 300 bytes at 6 fps and 19200 baud, about 1,800 at 115200 (budget = baud ÷ 10 ÷ fps × 0.95)
Serial speed	115200 baud, or 19200 with the 19200 receiver (2.4)
Tested on	a 1979 Apple II+, Super Serial Card II, Floppy Emu, Linux on a built-in serial port (details); and the simulator test suite

11.5 Regenerating the starter assets

```
python3 examples/make_examples.py
```

Rebuilds every file in `examples/`. Change the parameters in the script to make your own variations.